

**INSTITUTO FEDERAL GOIANO
SISTEMAS DE INFORMAÇÃO**

**BRUNO HENRIQUE PEREIRA SOUTO
CARLA MONTEIRO BRANDÃO
STEPHANE FREIRE DA SILVA SANTOS**

SISTEMA DE GESTÃO DE ACESSOS A MÓDULOS

BRUNO HENRIQUE PEREIRA SOUTO
CARLA MONTEIRO BRANDÃO
STEPHANE FREIRE DA SILVA SANTOS

SISTEMA DE GESTÃO DE ACESSOS A MÓDULOS

Trabalho Acadêmico apresentado ao Curso Superior de Tecnologia em Sistemas de Informação do Instituto Federal Goiano – Campus Urutaí, como requisito para avaliação na disciplina de Projeto de WEB II.

Orientador: Junio César de Lima.

SUMÁRIO

1	DESCRIÇÃO DO PROBLEMA	2
2	REQUISITOS E CASOS DE USO	3
2.1	Requisitos Funcionais	3
2.2	Diagrama de Casos de Uso	4
3	DIAGRAMAS DE CLASSE E MODELAGEM DE DADOS	5
3.1	Diagrama de Entidade-Relacionamento (DER)	6
4	PROJETO ARQUITETURAL E EXPLICAÇÃO DA IMPLEMENTAÇÃO	7
4.1	Divisão em Camadas Aplicada	7
4.2	Padrões de Projeto Utilizados	8
4.3	Estratégia de Performance e Decisão Arquitetural de Caching	8
4.4	Infraestrutura de Segurança e Filtro JWT	9
5	TESTES AUTOMATIZADOS E GARANTIA DA QUALIDADE	9
5.1	Testes de Unidade (Model)	10
5.2	Testes de Regras de Negócio (Service)	10
5.3	Testes de Integração (Controller)	10
6	CONCLUSÃO	11

1 DESCRIÇÃO DO PROBLEMA

No cenário corporativo moderno, a segurança da informação e a governança de dados tornaram-se pilares indispensáveis para a sobrevivência e a conformidade regulatória das empresas. O controle manual de privilégios de acesso a sistemas críticos expõe as organizações a sérios riscos de segurança, como vazamento de informações confidenciais, elevação indevida de privilégios e inconformidades em auditorias internas e externas.

Para contextualizar o desenvolvimento do sistema proposto, foi considerada uma empresa fictícia de grande porte chamada **LogiTech Distribuidora**. A LogiTech opera com uma infraestrutura de software integrada dividida em múltiplos módulos de negócios estratégicos, a saber:

- **Portal do Colaborador:** Módulo central de interface e serviços para funcionários;
- **Relatórios:** Ferramentas de auditoria e BI corporativo;
- **Financeiro:** Processamento de fluxos de caixa, faturamento e balanços;
- **Recursos Humanos:** Gerenciamento de folhas de pagamento e dados sensíveis de funcionários;
- **Estoque:** Controle físico de ativos, cadeia de suprimentos e logística;
- **Compras:** Homologação de fornecedores e ordens de aquisição.

O problema central identificado na LogiTech Distribuidora reside na ausência de uma plataforma centralizada e automatizada para a intermediação e auditoria de concessão de acessos. Sem uma ferramenta dedicada, as solicitações trafegam de forma descentralizada por e-mails ou planilhas físicas, resultando em lentidão operacional, sobrecarga de gestores e perda de rastreabilidade temporal (quem concedeu o acesso, para qual módulo, por quanto tempo e sob qual justificativa).

O desenvolvimento do sistema **Gatekeeper API** tem como objetivo sanar essas falhas estruturais, funcionando como um ecossistema Back-end robusto focado em mitigar riscos de segurança através do controle rigoroso de acessos por tempo limitado. A solução permite que usuários com privilégios básicos realizem solicitações formais detalhando justificativas operacionais e prazos estritos em dias. Essas solicitações são encaminhadas automaticamente aos gestores responsáveis pelo respectivo módulo para avaliação (aprovação ou rejeição) por meio de pareceres técnicos, garantindo auditoria ponta a ponta e governança em tempo real.

2 REQUISITOS E CASOS DE USO

A engenharia de requisitos do Gatekeeper API foi projetada para estabelecer barreiras claras de responsabilidade. O sistema define três perfis funcionais principais organizados hierarquicamente: `ROLE_ADMIN`, `ROLE_GESTOR` e `ROLE_COMUM`.

2.1 Requisitos Funcionais

- **RF01 – Gerenciar Usuários:** Permitir o cadastro, consulta, remoção, alteração e gerenciamento das informações dos usuários da aplicação.
- **RF02 – Gerenciar Departamentos:** Possibilitar o cadastro, remoção, consulta e alteração dos departamentos existentes na organização.
- **RF03 – Gerenciar Módulos:** Disponibilizar recursos para gerenciamento dos módulos do sistema, incluindo cadastro, consulta e manutenção das unidades de acesso disponíveis.
- **RF04 – Gerenciar Perfis:** Permitir a criação, consulta e atualização dos perfis de acesso dos usuários, definindo seus níveis de permissão.
- **RF05 – Atribuir e Listar Responsáveis por Módulo:** Possibilitar a associação de responsáveis aos módulos e a consulta dos responsáveis cadastrados para cada módulo.
- **RF06 – Abrir e Listar Solicitações:** Disponibilizar a abertura de solicitações de acesso aos módulos e a consulta do histórico de solicitações realizadas pelos usuários.
- **RF07 – Emitir e Listar Pareceres:** Permitir a análise das solicitações pelos responsáveis dos módulos, possibilitando a emissão de pareceres de aprovação ou rejeição.
- **RF08 – Consultar Notificações:** Oferecer aos usuários acesso às notificações referentes às alterações de status das solicitações.
- **RF09 – Consultar Histórico de Acessos:** Disponibilizar a consulta dos registros de acessos concedidos, contendo informações sobre período de validade e usuário autorizado.
- **RF10 – Gerenciar FAQ:** Permitir o cadastro, remoção, consulta e manutenção das perguntas frequentes disponíveis aos usuários.
- **RF11 – Autenticação de Usuários:** Realizar a autenticação dos usuários, garantindo o acesso às funcionalidades conforme o perfil de permissão associado.

2.2 Diagrama de Casos de Uso

O diagrama de casos de uso representa as interações físicas dos atores (*Colaborador Comum*, *Gestor* e *Administrador*) com as fronteiras de serviços disponibilizadas pela API Gatekeeper, ilustrando as rotas seguras e dependências de inclusão no login.

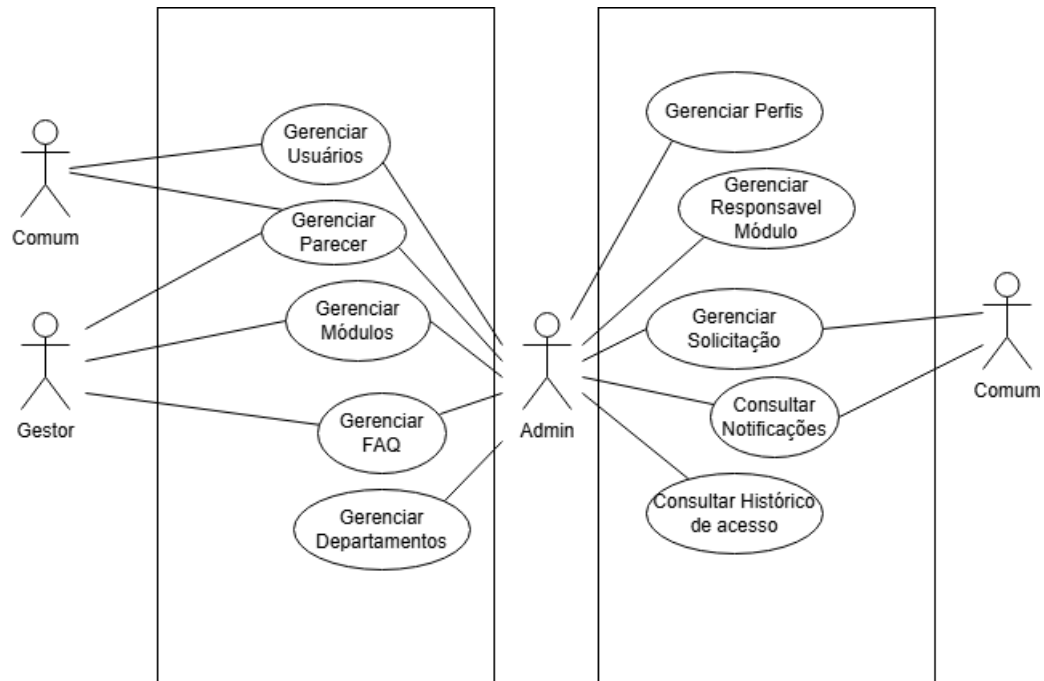


Figura 1: Diagrama de Casos de Uso

3 DIAGRAMAS DE CLASSE E MODELAGEM DE DADOS

A modelagem de dados e a arquitetura de classes do sistema foram elaboradas seguindo preceitos de POO (Programação Orientada a Objetos) e mapeamento objeto-relacional (ORM) com o framework JPA/Hibernate.

O modelo estrutural apresenta os seguintes relacionamentos entre as classes:

- **Associações Simples:** Perfil – Usuário; Usuário – Histórico de Acesso; Usuário – Responsável por Módulo.
- **Agregações:** Departamento – Usuário; Solicitação – Módulo.
- **Composições:** Módulo – FAQ; Responsável por Módulo – Módulo; Solicitação – Parecer; Parecer – Notificação.

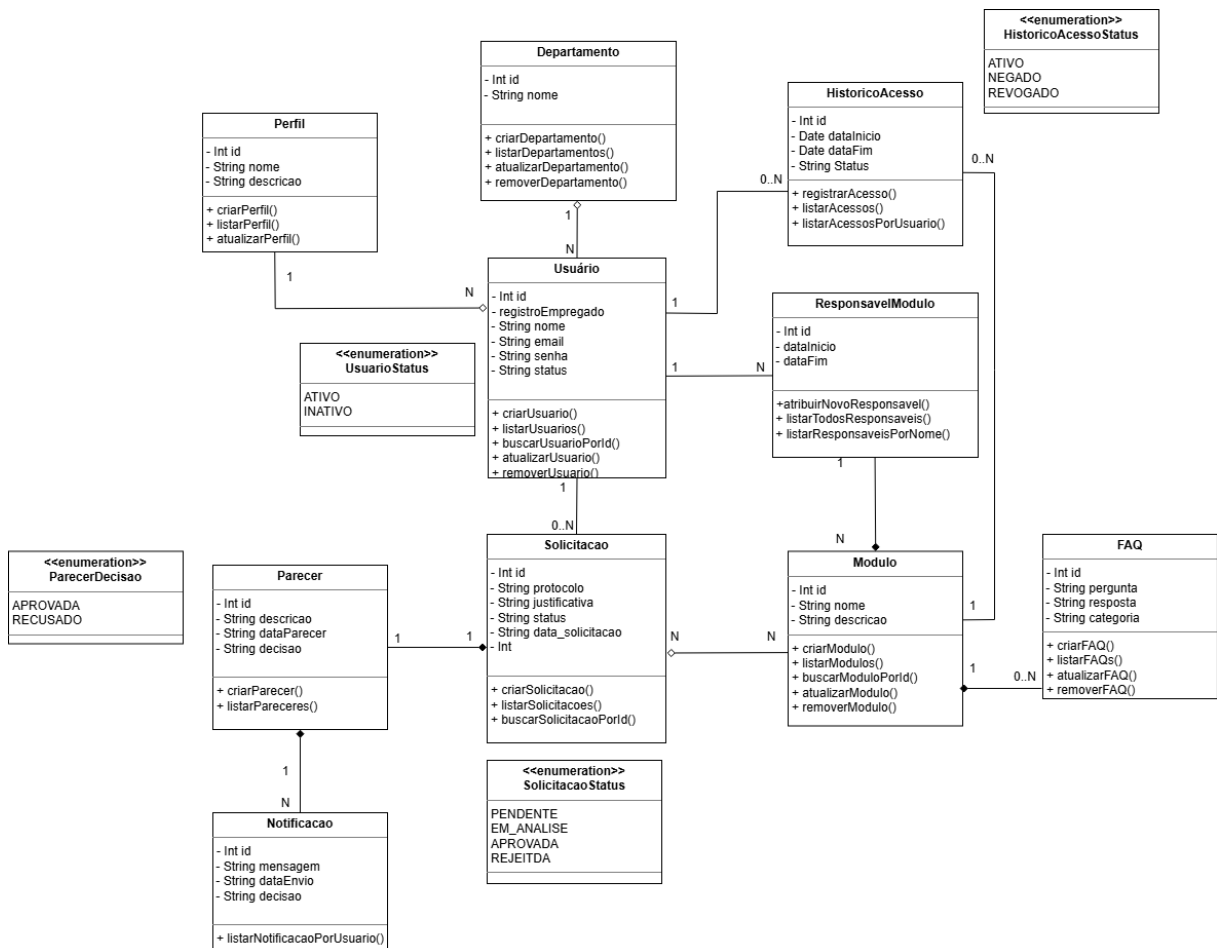


Figura 2: Diagrama de Classes

3.1 Diagrama de Entidade-Relacionamento (DER)

O modelo físico de dados reflete o esquema relacional gerado no banco de dados da LogiTech Distribuidora.

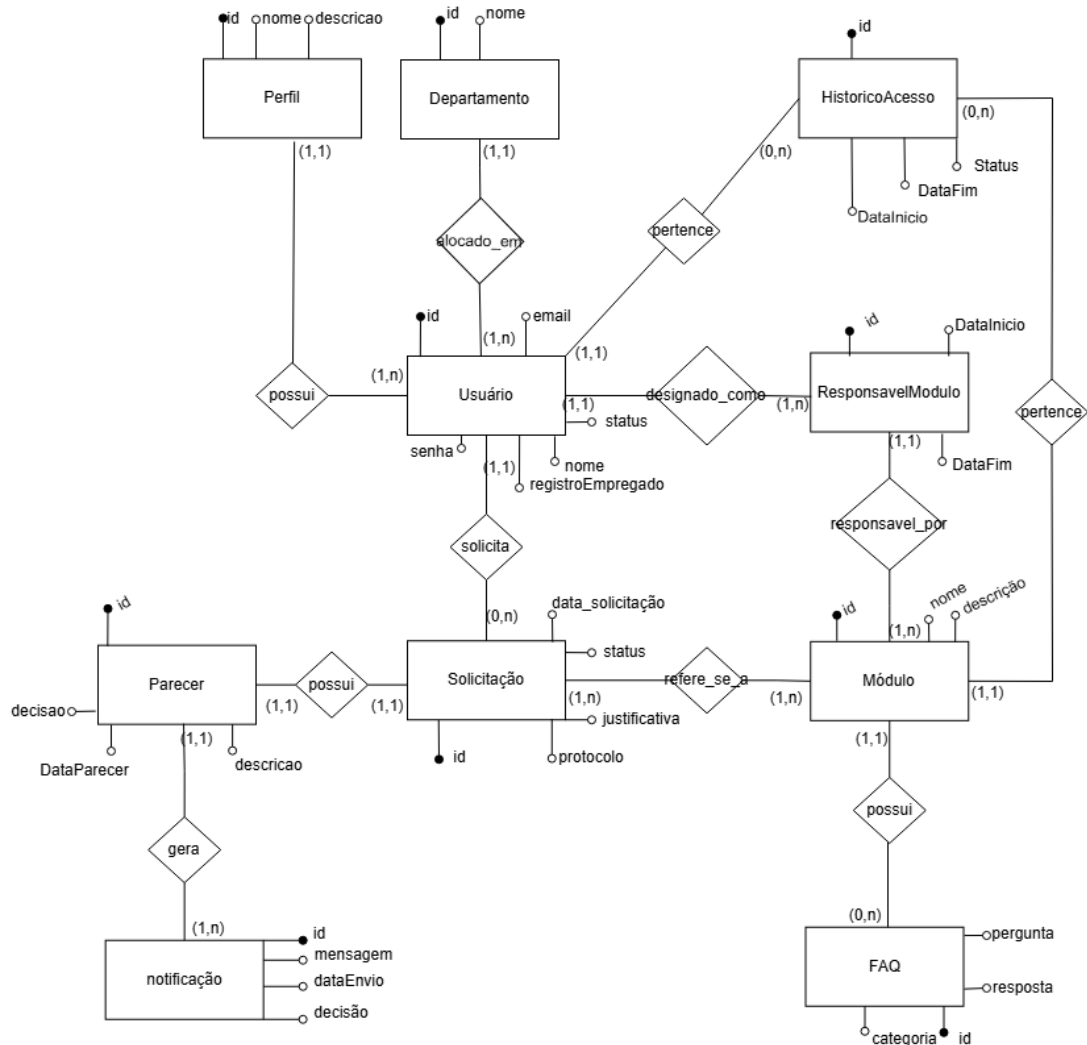


Figura 3: DER

4 PROJETO ARQUITETURAL E EXPLICAÇÃO DA IMPLEMENTAÇÃO

O Gatekeeper API foi estruturado utilizando uma arquitetura em camadas, composta principalmente pelas camadas de Controller, Service, Repository e Model. Além dessas camadas principais, o projeto possui componentes auxiliares, como DTOs, Mappers, configurações, tratamento de exceções e mecanismos de segurança, responsáveis por organizar funcionalidades específicas da aplicação.

4.1 Divisão em Camadas Aplicada

A aplicação foi estruturada utilizando uma arquitetura em camadas (*Layered Architecture*), organizando as responsabilidades do sistema em componentes separados. As principais camadas implementadas no projeto são:

1. **Camada de Controladores (Controller):** Representa a porta de entrada da aplicação, sendo responsável pelo recebimento das requisições HTTP, validação dos dados enviados pelo cliente e retorno das respostas. Nesta camada são definidos os endpoints da API, os códigos de status HTTP e as regras de autorização utilizando anotações como `@PreAuthorize`. Também são disponibilizadas as documentações da API por meio do OpenAPI/Swagger.
2. **Camada de Serviços (Service):** Responsável pela implementação das regras de negócio da aplicação. Essa camada realiza a comunicação entre os controladores e os repositórios, coordenando operações como criação de solicitações, análise de pareceres, geração de notificações e atualização do histórico de acessos. Além disso, utiliza operações transacionais através da anotação `@Transactional` para garantir a consistência dos dados.
3. **Camada de Domínio (Model):** Contém as entidades que representam os principais objetos do sistema e seus relacionamentos, como `Usuario`, `Modulo`, `Solicitacao`, `Parecer` e `Notificacao`. Essas classes são responsáveis pelo mapeamento objeto-relacional utilizando JPA/Hibernate, definindo atributos, associações e regras relacionadas à estrutura dos dados persistidos.
4. **Camada de Acesso a Dados (Repository):** Responsável pela comunicação com o banco de dados através das interfaces do Spring Data JPA (`JpaRepository`). Essa camada permite realizar operações de persistência e consultas sobre as entidades do sistema, reduzindo a necessidade de implementação manual de comandos SQL.

4.2 Padrões de Projeto Utilizados

A robustez do Gatekeeper API decorre da aplicação sinérgica de padrões consolidados de engenharia de software:

- **Data Transfer Object (DTO):** Segrega os dados de entrada das estruturas de exibição, divididos estritamente em payloads de requisição (`RequestDTO`) e respostas (`ResponseDTO`). Esse isolamento previne de forma categórica vulnerabilidades severas de segurança conhecidas como *Mass Assignment* (ou preenchimento em massa de propriedades restritas como IDs e Status pelo cliente HTTP). Além disso, o sistema utiliza recursos modernos de Java `records` para DTOs imutáveis de autenticação (`DadosAutenticacao`) e respostas rápidas (`DadosTokenJWT`, `FaqResponseDTO`).
- **Data Mapper (MapStruct):** A tradução tipada entre modelos persistentes e DTOs é efetuada em tempo de compilação por meio de interfaces anotadas com `@Mapper(componentModel = "spring")`. A ferramenta realiza o achatamento de dados de forma performática, capturando atributos profundos (ex: nome do departamento dentro do objeto mapeado do usuário) e convertendo-os em strings planas antes do envio ao cliente corporativo.
- **Value Object (VO):** Implementado na estruturação do atributo de contato do usuário (`Email`). Sendo um objeto imutável encapsulado com validação estrutural via expressões regulares (Regex), ele impede o nascimento de um usuário portando um endereço eletrônico inválido no ecossistema de software, delegando o controle de igualdade interna aos métodos `equals` e `hashCode`.

4.3 Estratégia de Performance e Decisão Arquitetural de Caching

A aplicação emprega o mecanismo de cache abstrato do Spring alimentado pelo motor de alta performance Caffeine. Contudo, a decisão de aplicação foi conduzida sob o princípio rigoroso do **YAGNI** (*You Aren't Gonna Need It*).

Entidades altamente transacionais que sofrem mutações constantes em curtos intervalos de tempo, como `Solicitacao`, `Parecer` e `Notificacao`, foram deliberadamente mantidas fora da memória de cache. A inclusão de cache nestes módulos exigiria políticas agressivas de invalidação cruzada em massa (`@CacheEvict(allEntries = true)`) a cada alteração de status por gestores, gerando sobrecarga computacional de limpeza de memória e introduzindo o risco de dados defasados na tela do colaborador. Por outro lado, o cache foi estrategicamente mantido em tabelas estruturais estáveis e de leitura exaustiva, como as consultas de `DepartamentoService` e listagens de `UsuarioService`,

PerfilService, maximizando o rendimento e poupando conexões ativas com o banco de dados relacional.

4.4 Infraestrutura de Segurança e Filtro JWT

A arquitetura de segurança é stateless. As configurações gerais em SecurityConfigurations centralizam o fluxo, desabilitando o *Cross-Site Request Forgery* (csrf) por conta do modelo desacoplado de tokens e definindo uma hierarquia de papéis explícita através do componente RoleHierarchy. O algoritmo de herança estabelece que:

$$\text{ROLE_ADMIN} > \text{ROLE_GESTOR} > \text{ROLE_COMUM} \quad (1)$$

Toda requisição direcionada a endpoints protegidos é retida pelo componente SecurityFilter, uma subclasse especializada de OncePerRequestFilter. O ciclo de interceptação compreende os seguintes passos sequenciais automatizados:

```
1 // Pseudocódigo estrutural do fluxo do filtro de segurança JWT
2 String token = recuperarToken(request);
3 if (token != null) {
4     String subjectEmail = tokenService.getSubject(token); // Valida
    assinatura HMAC256
5     UserDetails usuario = repository.findByEmail(subjectEmail);
6     var authentication = new UsernamePasswordAuthenticationToken(
7         usuario, null, usuario.getAuthorities()
8     );
9     SecurityContextHolder.getContext().setAuthentication(authentication)
10 ;
11 }
12 filterChain.doFilter(request, response);
```

Se a autenticação falhar ou o cabeçalho Authorization estiver corrompido, o manipulador customizado de exceções intercepta a falha através de um componente AuthenticationEntryPoint acoplado à cadeia, devolvendo ao cliente um objeto JSON customizado com status HTTP 403 Forbidden formatado de maneira uniforme, blindando a exibição de rastros internos ou páginas genéricas de erro do servidor web.

5 TESTES AUTOMATIZADOS E GARANTIA DA QUALIDADE

Para garantir que o sistema Gatekeeper API funcione corretamente e sem erros, foi criada uma suíte de testes automatizados seguindo a ideia da Pirâmide de Testes. Os testes foram desenvolvidos usando JUnit 5, AssertJ (para facilitar as verificações) e Mockito (para simular dependências). Além disso, todos os testes seguem o padrão AAA

(Arrange, Act, Assert), que organiza o código em três etapas: preparar os dados, executar a ação e verificar o resultado.

5.1 Testes de Unidade (Model)

Nos testes de unidade, o foco foi validar o comportamento das entidades do sistema de forma isolada. Por exemplo, na classe `Solicitacao`, foram testados os métodos básicos como getters e setters, garantindo que os dados sejam armazenados e recuperados corretamente.

Também foi verificado se os relacionamentos com outras classes, como `Usuario` e `Modulo`, estão funcionando como esperado. Esses testes ajudam a evitar erros simples que podem quebrar partes importantes do sistema.

5.2 Testes de Regras de Negócio (Service)

Na camada de serviço, onde fica a lógica principal do sistema, os testes foram feitos utilizando o Mockito para simular o comportamento do banco de dados. Assim, foi possível testar tudo de forma mais rápida e sem depender de uma base real.

Foram criados testes tanto para cenários de sucesso quanto para erros. Por exemplo, quando um usuário ou módulo não existe, o sistema deve lançar uma exceção e impedir que a solicitação seja salva. Isso garante que dados inválidos não sejam persistidos.

5.3 Testes de Integração (Controller)

Para testar a parte da API, foram feitos testes de integração usando `@WebMvcTest` e a ferramenta MockMvc. Esses testes simulam requisições HTTP (como GET e POST) e verificam se as respostas estão corretas, incluindo status e retorno em JSON.

Também foram testadas as regras de segurança com `@WithMockUser`, simulando usuários com diferentes permissões. Assim, foi possível garantir que apenas usuários autorizados conseguem acessar determinadas rotas.

No geral, esses testes ajudam a garantir que o sistema funcione corretamente em diferentes situações, aumentando a confiabilidade e facilitando a manutenção do código.

6 CONCLUSÃO

O desenvolvimento do **Gatekeeper API** permitiu aplicar os conceitos de engenharia de software propostos para a construção de uma API Back-end organizada e estruturada. O projeto foi desenvolvido seguindo uma arquitetura em camadas, separando as responsabilidades entre controladores, serviços, repositórios e modelos de domínio, proporcionando uma melhor organização do código e facilitando a manutenção da aplicação. A implementação buscou atender aos requisitos definidos para o sistema de gerenciamento e controle de acessos da **LogiTech Distribuidora**.

Durante o desenvolvimento, foram aplicados recursos e práticas importantes do ecossistema Java, como o Spring Security para controle de autenticação e autorização, JWT para gerenciamento de sessões, DTOs e Value Objects para organização e validação dos dados, além do uso de JPA/Hibernate para o mapeamento dos relacionamentos entre as entidades e persistência no banco de dados. Também foram utilizados testes automatizados, documentação da API com Swagger, implementação de HATEOAS e mecanismos de cache, conforme os requisitos estabelecidos para o projeto.

Como melhorias futuras, identificaram-se alguns pontos de evolução para o sistema, como a implementação de validações mais restritivas de acesso entre usuários e gestores, garantindo que cada perfil visualize e altere somente informações compatíveis com suas permissões. Também podem ser adicionados novos métodos para atualização de responsáveis por módulos, consultas específicas de pareceres para gestores, tratamento adequado de respostas HTTP como **404 Not Found** para recursos inexistentes e ajustes nas regras de consulta de solicitações para impedir acesso indevido a dados de outros usuários. Além disso, a expansão das funcionalidades de integração e comunicação assíncrona pode contribuir para a evolução da plataforma.